

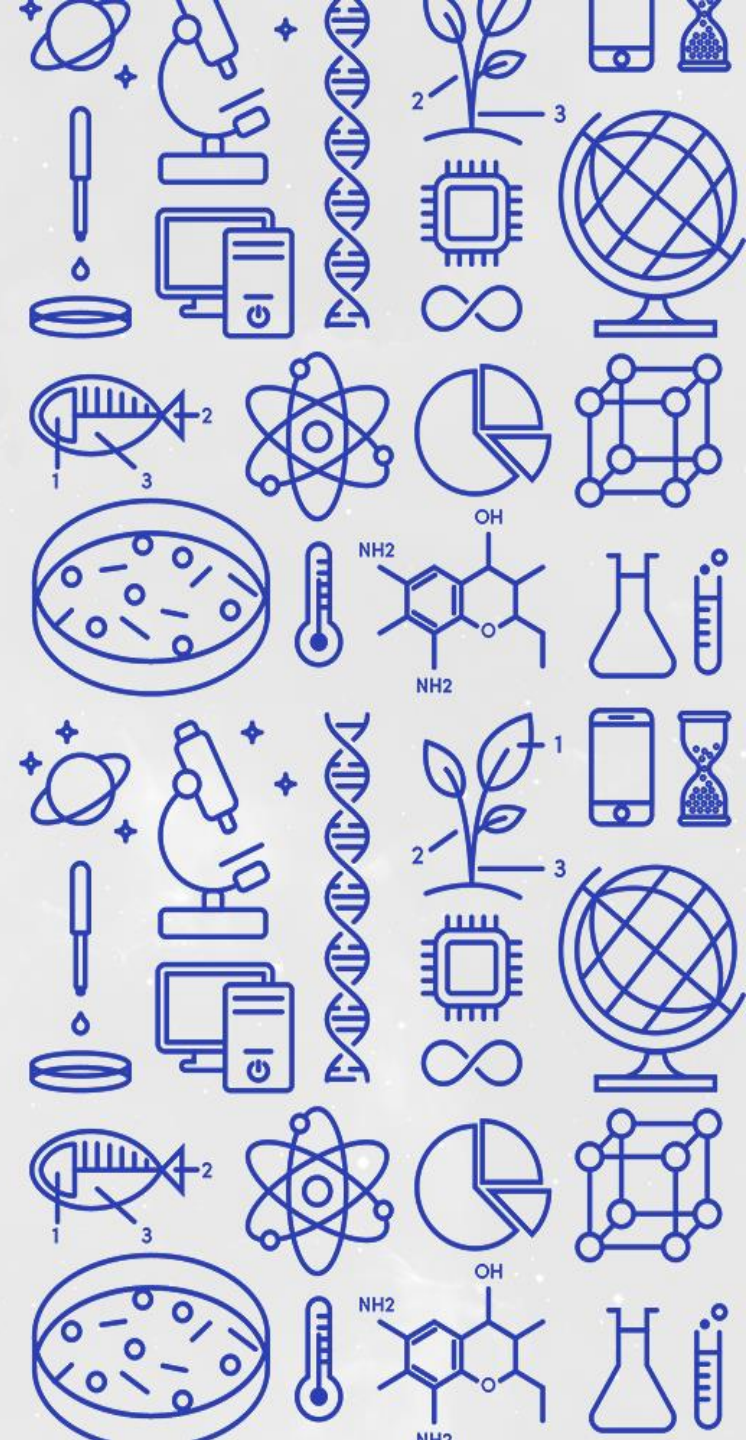
Ciências
ULisboa

Informática

Sistemas Distribuídos na Web com REST

José Cecílio

AD - TP05 | ©DI, Ciências, ULisboa



1. REST: REpresentational State Transfer

2. Métodos REST

3. APIs REST

1. REST: REpresentational State Transfer

- **REST** é um **estilo arquitetural** para aplicações distribuídas
- **Características** importantes:
 - Muito **simples**
 - **Independente** de **plataforma**
 - **Independente** de **linguagem** de programação
 - Baseado em **standards** (normalmente corre sobre o HTTP)
 - Convive pacificamente com *firewalls*
- **Diferença fundamental para o RPC/Web Services (SOAP):**
 - **Não** é possível definir **métodos arbitrários** para os “objetos distribuídos”, aqui chamados “recursos”
 - Os **métodos** suportados **são** basicamente o padrão **CRUD** (*Create/Read/Update/Delete*)
 - Apesar desta aparente limitação, não há nada que se possa fazer com RPC e que não possa ser feito com REST

1. REST: Conceitos Fundamentais

- **Recursos**

- Qualquer coisa suficientemente importante para ser referenciada por uma **URL (o nome do recurso)**
- Uma **aplicação REST** é normalmente organizada num **conjunto de recursos** que são **endereçados** e tem o seu estado lido e modificado
- Normalmente um **recurso** é algo que pode ser armazenado e **representado** através de uma **sequência de bits**
- Exemplos: um documento, uma versão de um software, um tweet, a lista de utilizadores de uma base de dados, etc.

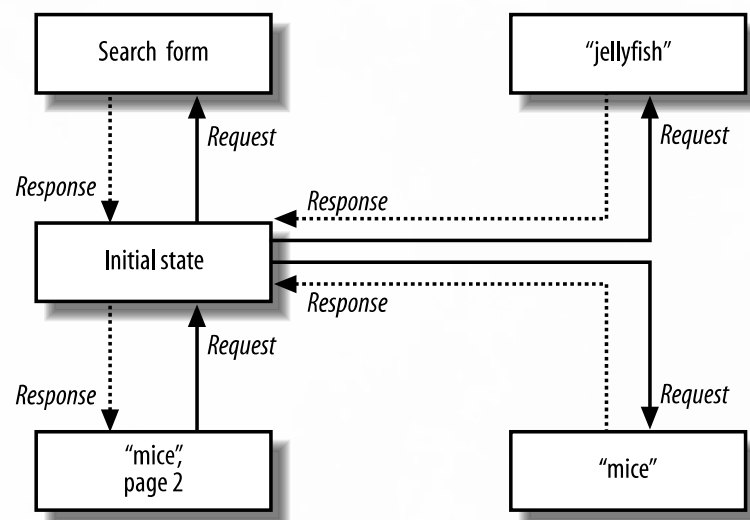
- **Representações**

- **Recursos são ideias, não dados.**
No entanto, o **estado** de um **recurso** pode ser **lido** ou **modificado**
- Isto implica que os recursos têm de ser representados de alguma forma útil
- Representações podem conter **links para outros recursos**
- Exemplos de representações: HTML, XML, JSON, etc.

1. REST: Restrições Arquiteturais

- **Cliente-servidor**
 - Melhora a separação de papéis e responsabilidades
- **Stateless** – interações entre clientes e servidores são *stateless* (tipo UDP, não TCP)
 - Melhora a escalabilidade
- **Cache** – recursos podem ser mantidos em cache
 - Melhora a eficiência no uso da rede
- **Identificação de recursos**
 - Através de URLs
- **Manipulação de recursos através de representações**
 - Recursos são representados como sequências de bits
- **Mensagens auto-descritivas**
 - Para um humano ou aplicação
- **Hipermédia como motor do estado da aplicação (*Connectedness*)**
 - Representações de recursos contém links, e o estado da aplicação pode ser mantido pelo cliente durante as interações

1. REST: Ausência de estado na comunicação (*Stateless*)



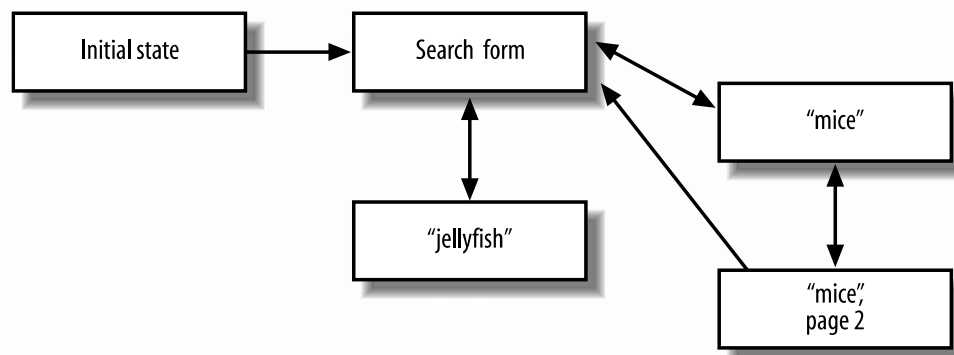
Motor de busca sem estado

(baseado em HTTP)

Exemplos de buscas:

`www.google.com/search?q=mice`

`www.google.com/search?q=mice&start=10`



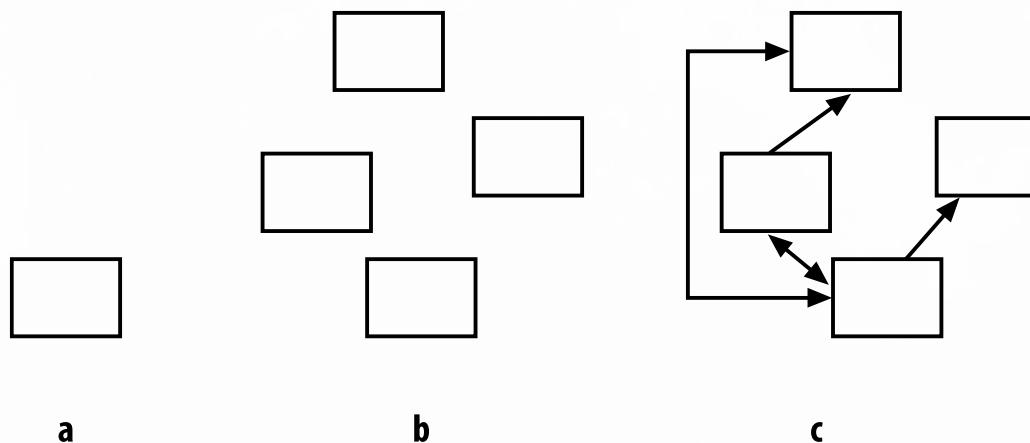
Motor de busca com estado

(neste caso o servidor conhece o estado do cliente –

tem de usar um protocolo *stateful*, como o FTP, muito mais complexo que o HTTP)

1. REST: Evolução de um aplicação

- Três exemplos que expõem a mesma funcionalidade, mas com usabilidade maior quanto mais para a direita
 - a) Típico serviço estilo RPC. Expõe tudo através de uma única URI. Não é endereçável, nem conectado
 - b) Serviço REST-RPC ou RESTful: Endereçável, mas não conectado, pois não possui ligações entre os recursos
 - c) Serviço completo RESTful: Endereçável e bem conectado, pois os recursos estão interligados uns aos outros.



- Uma das características mais importantes do REST é sua **interface uniforme**
- Independentemente da aplicação, os métodos suportados pelos recursos serão sempre os mesmos, correspondentes aos métodos do HTTP:
 - **POST, GET, PUT e DELETE**
 - **HEAD e OPTIONS**
 - PATCH, LINK e UNLINK
- Dois conceitos importantes com relação aos métodos REST:
 - **Safety:** Um **método** é dito ser **safe** (ou seguro), se ele **não altera o estado de um recurso** (i.e., ele pode ser executado zero ou mais vezes com o mesmo efeito). Os métodos de leitura (GET e HEAD) são **safe**.
 - Métodos safe podem ser executados sobre recursos desconhecidos
 - **Idempotência:** Um método é **idempotente** se sua **execução uma ou mais vezes tem exatamente o mesmo efeito**. Os métodos de leitura, bem como métodos como PUT e DELETE, são idempotentes.
 - Métodos **idempotentes (e safe)** são **úteis** em **redes pouco fiáveis**, pois podem ser retransmitidos sem maiores constrangimentos.

2. Método GET

- Usado para **ler o estado** de um recurso
 - O cliente envia a URL de um recurso
 - O servidor retorna uma representação do recurso
- Exemplo:
 - Pedido de leitura ao recurso <http://www.mysite.com/api/myresource>:

```
GET /api/myresource HTTP/1.1  
Host: www.mysite.com
```

- Resposta

```
HTTP/1.1 200 OK  
Content-type: application/json  
  
{ "name": "value",  
  "photo link": "http://www.mysite.com/api/photo001" }
```

200 é o código da operação bem sucedida.

(Representação em JSON – mais na próxima aula)

- Lembrar que o GET é um método safe.

2. Método DELETE

- Usado para **destruir** um **recurso**
 - O cliente envia a URL de um recurso
 - O servidor confirma se o recurso foi destruído ou não
- Exemplo:
 - Pedido de destruição do recurso `http://www.mysite.com/api/myresource`:

```
DELETE /api/myresource HTTP/1.1  
Host: www.mysite.com
```

- Respostas típicas

```
HTTP/1.1 204 No Content  
HTTP/1.1 200 OK  
HTTP/1.1 202 Accepted
```

Está apagado!

Será apagado em breve!

- Depois de um recurso ser apagado, o GET retorna

```
HTTP/1.1 404 Not Found  
HTTP/1.1 410 Gone
```
- Lembrar que DELETE é um método idempotente.

2. Método PUT

- Usado para **atualizar o estado** de um recurso
 - O cliente envia a URL e uma nova representação de um recurso
 - O servidor confirma (ou não) a modificação
- Exemplos:
 - Pedido de alteração do recurso <http://www.mysite.com/api/myresource>:

```
PUT /api/myresource HTTP/1.1
Content-type: application/json

{"name": "value2",
 "photo_link": "http://www.mysite.com/api/photo002"}
```

- Respostas

```
HTTP/1.1 200 OK
HTTP/1.1 204 No Content
```

Modificação efetuada!

- PUT é idempotente, e pode também ser usado para criar um novo recurso caso conheça-se a URL que seria atribuída a ele.

2. Método POST (to append)

- O POST é um método genérico, mas o seu uso mais comum consiste na **criação de novos recursos**
 - No cliente, a representação de um novo recurso a ser criado
 - No servidor, retorna a localização do novo recurso
- Exemplos:
 - Pedido de criação de um novo recurso em `http://www.mysite.com/api/`:

```
POST /api/ HTTP/1.1
Content-type: application/json

{"name":"value3",
 "photo_link":"http://www.mysite.com/api/photo003"}
```
 - Resposta:

```
HTTP/1.1 201 Created
Location: http://www.mysite.com/api/newresource
```
- POST não é idempotente (a princípio).

URL do novo recurso.

2. Método HEAD

- Método similar ao GET, mas não **retorna** uma representação (*content body*), **apenas os cabeçalhos**

- O cliente envia a URL de um recurso
- O servidor retorna os cabeçalhos relativos ao recurso

- Exemplo:

- Pedido:

```
HEAD /api/myresource HTTP/1.1
Accept:application/json
```

- Resposta:

```
HTTP/1.1 200 OK
Content-type: application/json
ETag: "dd12312esdf34234234ab123"
Last-Modified: Thu, 24 Jan 2015 18:40:42 GMT
Date: Thu, 24 Jan 2015 19:14:11 GMT
```

Ver docs sobre
headers HTTP.

- HEAD é um método *safe*.

2. Método OPTIONS

- Usado para **descobrir os mecanismos HTTP**
 - O cliente envia a URL de um recurso
 - O servidor retorna métodos suportados pelo recurso
- Exemplo:
 - Pedido das opções do recurso `http://www.mysite.com/api/myresource`:

```
OPTIONS /api/myresource HTTP/1.1
Host: www.mysite.com
```
 - Resposta:

```
HTTP/1.1 200 OK
Allow: GET PUT DELETE HEAD OPTIONS
```


- OPTIONS é um método safe.
- Apesar de ser uma boa ideia, esse método não é muito usado.
- A maioria das boas APIs respondem a um GET com um documento hipermédia com vários links e informações sobre a API.

2. Método PATCH

- Trata-se de um método extra (RFC 5789, de 2010), não pertencente ao HTTP nativo, usado para **atualizar o estado** de um recurso, mas **enviando apenas partes** da representação
 - O cliente envia a URL e uma representação das diferenças que devem ser aplicadas ao estado de um recurso
 - O servidor confirma (ou não) as modificações
- Exemplo (respostas similares ao PUT):
 - Pedido de alteração parcial do recurso:

```
PATCH /api/myresource HTTP/1.1
Content-type: application/json-patch+json

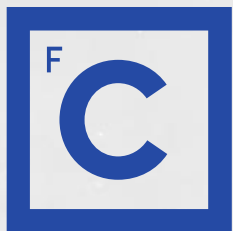
[{"op": "replace", "path": "/api/myresource",
  "photo_link": "http://www.mysite.com/api/photo003"}]
```



(Isso é patch-json, que suporta outras ops como add, test, remove, etc)
- Em princípio, PATCH não é safe nem idempotente.

- POST (overloaded)
 - Uso esperado do método POST (de acordo com a especificação do HTTP):
“Providing a block of data, such as the result of submitting a form, to a data handling process”
 - Ou seja, pode ser usado para modificar (ou até criar) um recurso
- LINK e UNLINK (também extensões modernas)
 - Operações usadas para **gerir links hipermédia** entre recursos
 - LINK associa uma URL a um recurso
 - UNLINK desassocia uma URL de um recurso
- CONNECT e TRACE
 - Usados para **lidar com proxies**, não são (normalmente) usados em serviços RESTfull
- COPY, MOVE, LOCK, entre outros, definidos na especificação WebDAV

- Além de uma forma relativamente simples de organizar um sistema distribuído, o REST é uma excelente forma de concretizar APIs abertas (ver OpenAPI, <https://www.openapis.org>)
- Alguns exemplos:
 - **Twitter API** (<https://dev.twitter.com/rest/public>):
 - A API não é completamente RESTful, os nomes de recursos são basicamente nomes de métodos
 - Os métodos usados são sempre GET e POST
 - **Amazon S3** (<http://docs.aws.amazon.com/AmazonS3/latest/API>):
 - Recursos: *Service* (GET), *Bucket* (GET, DELETE, HEAD e PUT) e *Object* (GET, DELETE, HEAD, OPTIONS, POST e PUT)
 - Representação: formato XML criado pela Amazon



Ciências
ULisboa

Informática

Sistemas Distribuídos na Web com REST

José Cecílio

AD - TP05 | ©DI, Ciências, ULisboa

